Prácticas de laboratorio Algoritmia

David Rico de los Bueis 2025/2026

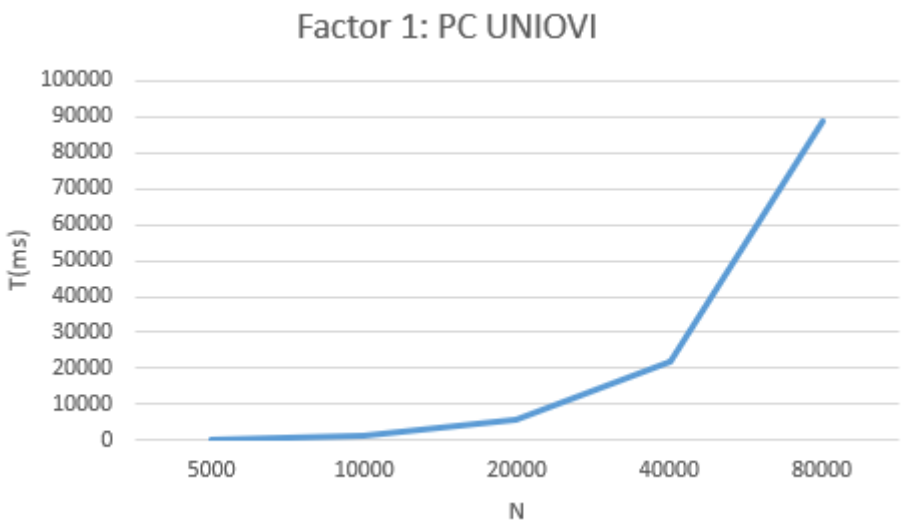
Contenido

Práctica 0 3
Práctica 1.1 10
Práctica 1.2 17
Práctica 2 19
Práctica 3 23
Práctica 4 29
Práctica 5 30
Práctica 6 31
Práctica 7 32

n	5000	10000	20000	40000	80000
T (ms)	335	1354	5472	22150	88892

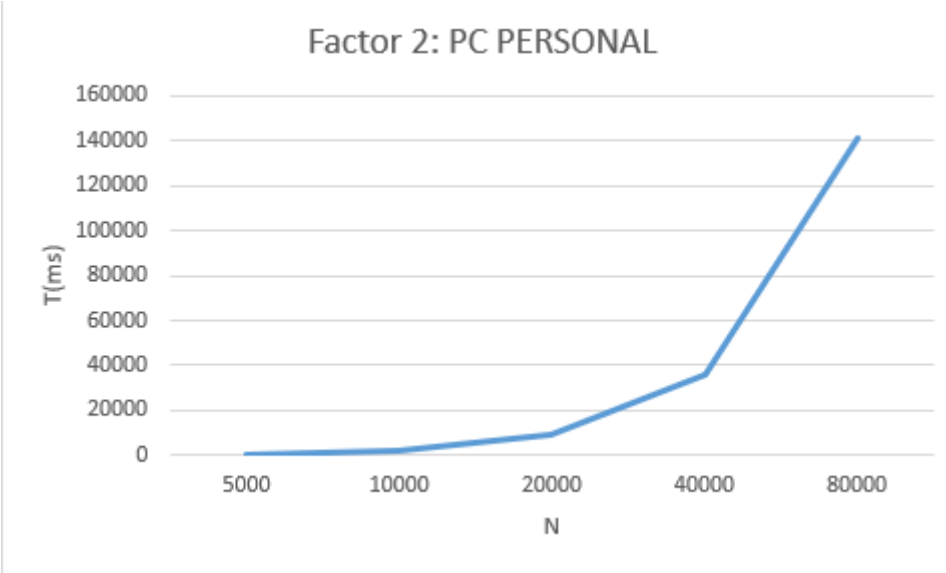
Práctica 0 David Rico UO306810

Factor 1



Factor 2

n	5000	10000	20000	40000	80000
T (ms)	545	2135	9056	35597	141203

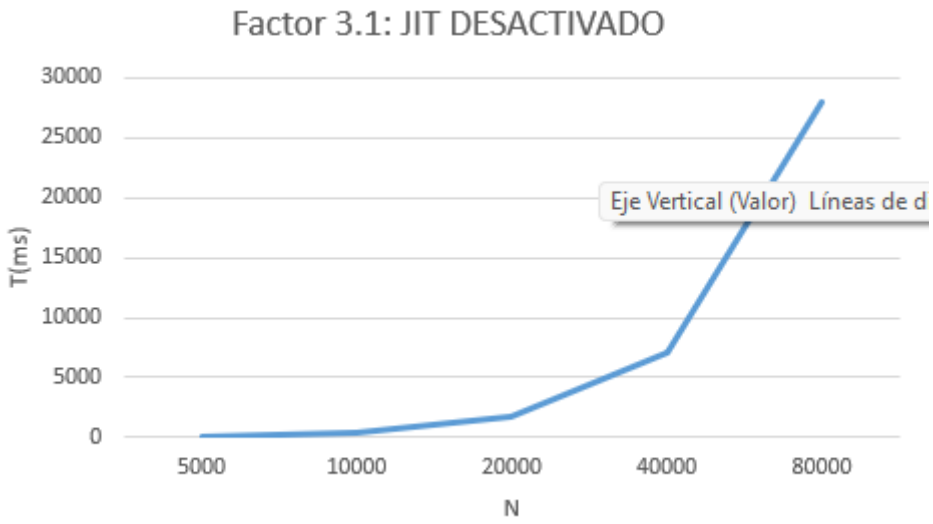


Procesador: Intel(R) Core(TM) i5-8400 CPU @ 2.80GHz (2.81 GHz)

Ram: 16,0 GB

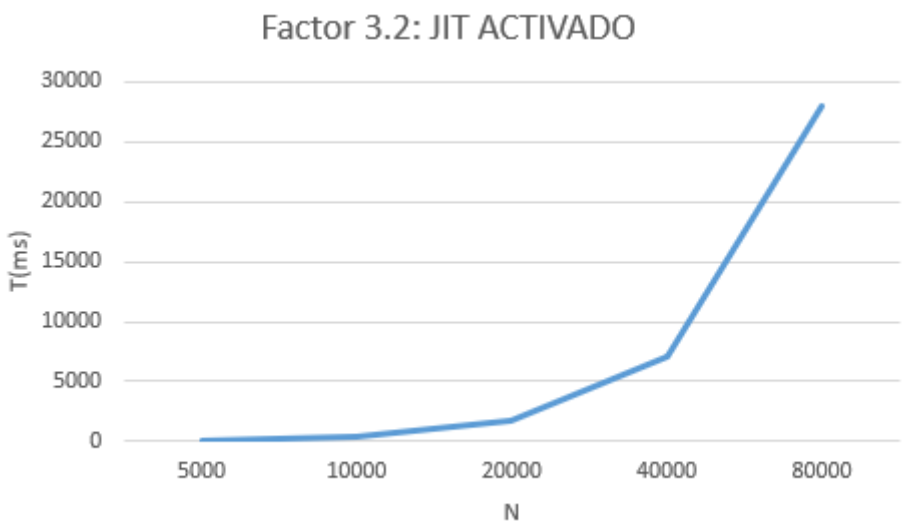
Factor 3.1 (ordenador personal)

n	5000	10000	20000	40000	80000
T (ms)	117	436	1756	7024	27918



Factor 3.2 (ordenador personal)

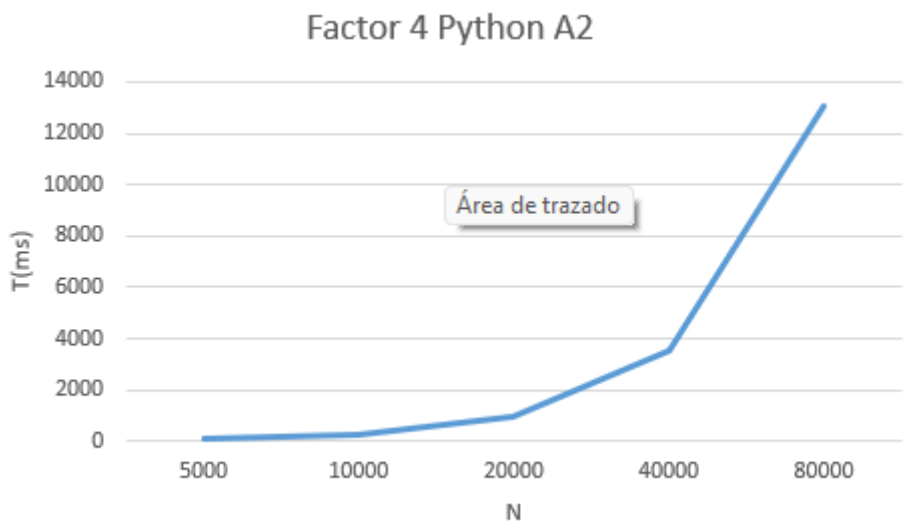
n	5000	10000	20000	40000	80000
T (ms)	36	109	413	1649	6690



Factor 4.Python

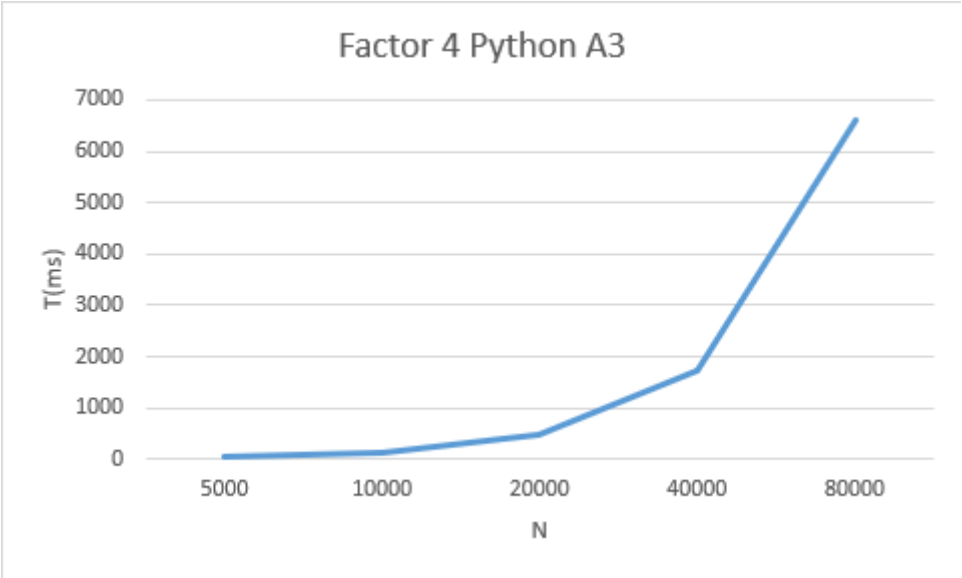
Python A2

n	5000	10000	20000	40000	80000
T (ms)	68	251	940	3553	13060



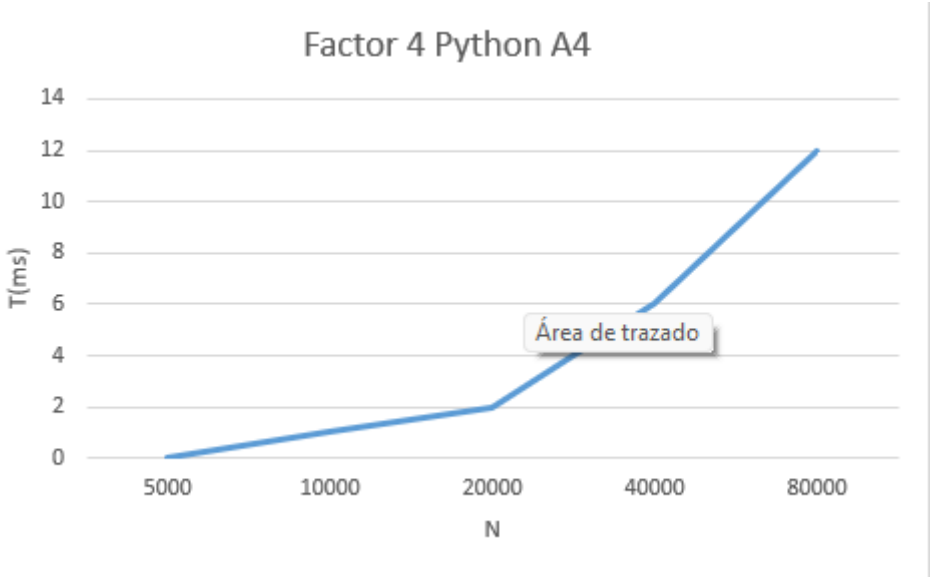
Python A3

n	5000	10000	20000	40000	80000
T (ms)	33	126	470	1732	6602



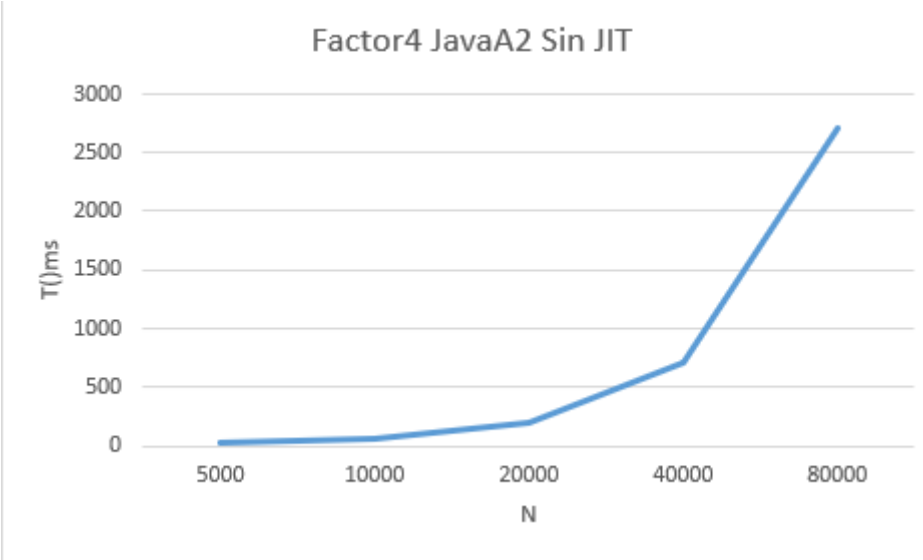
Python A4

n	5000	10000	20000	40000	80000
T (ms)	0	1	2	6	12



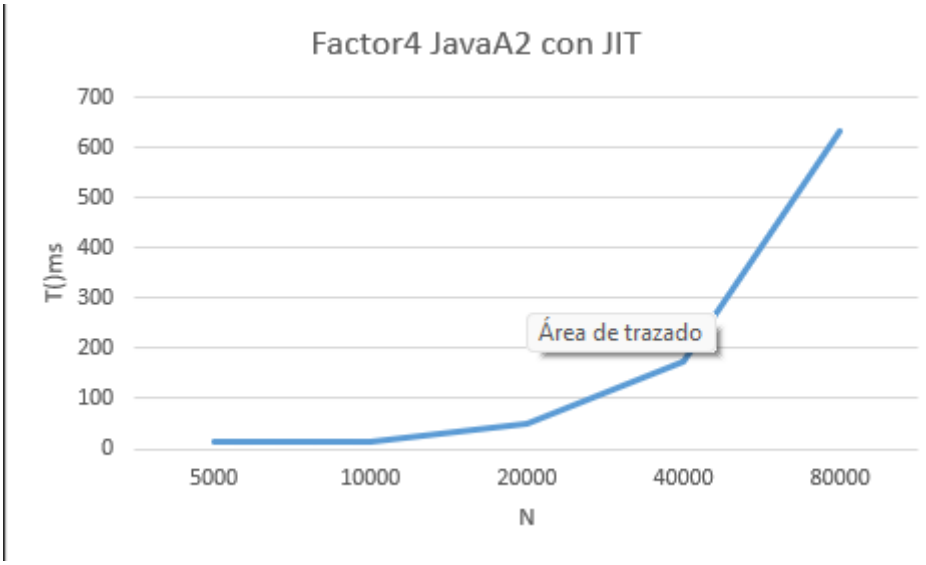
Java A2 (jit desactivado)

n	5000	10000	20000	40000	80000
T (ms)	20	53	193	717	2700



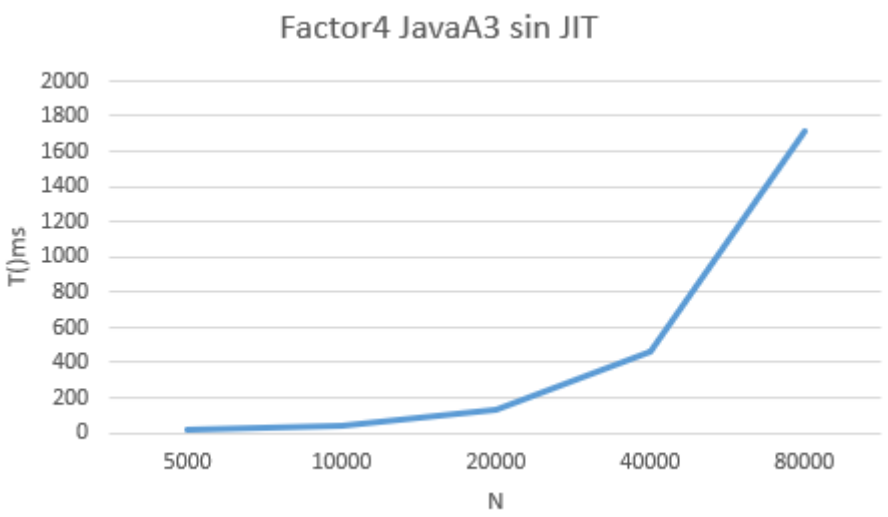
Java A2 (JIT activado)

n	5000	10000	20000	40000	80000
T (ms)	14	15	49	174	632



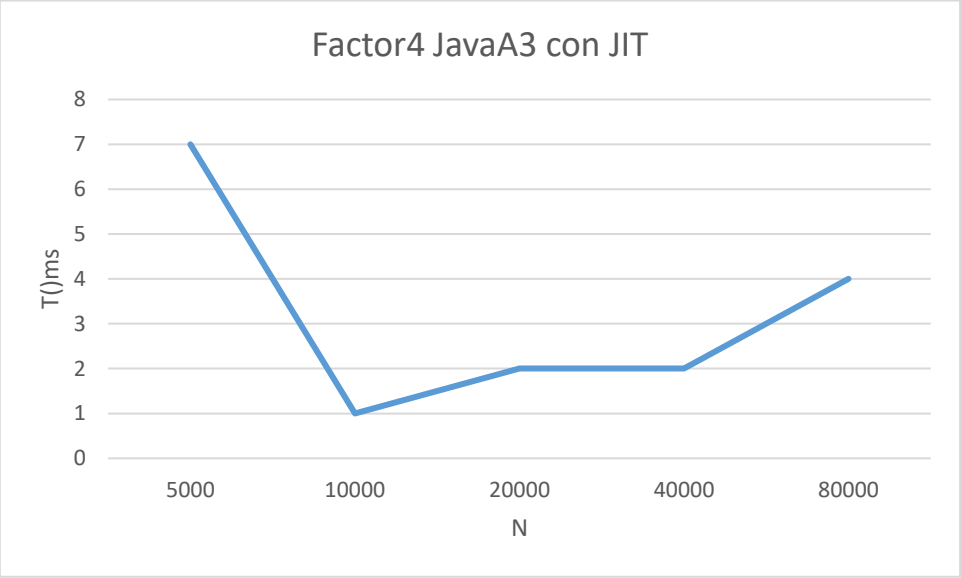
Java A3 (JIT desactivado)

n	5000	10000	20000	40000	80000
T (ms)	16	35	127	466	1719



Java A3 (JIT activado)

n	5000	10000	20000	40000	80000
T (ms)	12	7	23	85	310

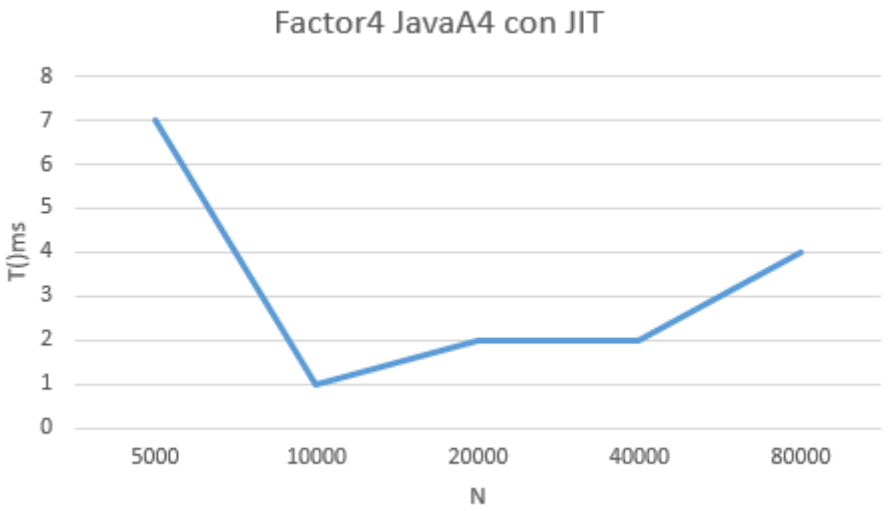


Java A4 (JIT desactivado)

n	5000	10000	20000	40000	80000
T (ms)	7	0	1	2	4

Java A4 (JIT activado)

n	5000	10000	20000	40000	80000
T (ms)	7	1	2	2	4



Práctica 1.1

(Puse capturas del PDF porque a juntar los words se me descuadraba toda las tablas.)

java -Xint p11.Vector4 1

N	T
10000	1
20000	0
40000	0
80000	1
160000	1
320000	3
640000	5
1280000	11
2560000	22
5120000	45
10240000	87
20480000	175
40960000	357
81920000	695

java -Xint p11.Vector4 10

N	T
10000	1
20000	1
40000	4
80000	7
160000	13
320000	28
640000	57
1280000	111
2560000	220
5120000	440
10240000	884
20480000	1750
40960000	3510
81920000	7023

java -Xint p11.Vector4 100

N	T
10000	9
20000	17
40000	35
80000	68
160000	134
320000	274
640000	547
1280000	1086
2560000	2201
5120000	4357
10240000	8652
20480000	17455
40960000	34872
81920000	69908

java -Xint p11.Vector4 1000

N	T
10000	85
20000	168
40000	334
80000	678
160000	1359
320000	2705
640000	5477
1280000	10946
2560000	21658
5120000	44104
10240000	87495
20480000	172186
40960000	349995
81920000	697507

java -Xint p11.Vector4 100000

N	T
10000	8326
20000	17398
40000	34513
80000	68993
160000	137210
320000	270531
640000	547811
1280000	1094734
2560000	2165845
5120000	4410391
10240000	8749612
20480000	17218623
40960000	34999562
81920000	69750721

java -Xint p11.Vector5 10

N	T
10000	1
20000	2
40000	4
80000	8
160000	16
320000	31
640000	63
1280000	120
2560000	238
5120000	490
10240000	949
20480000	1898
40960000	3816
81920000	7643

java -Xint p11.Vector6 1

N	T
10000	692
20000	2797
40000	11239
80000	45246
160000	182280
320000	734343
640000	Fdt
1280000	Fdt
2560000	Fdt
5120000	Fdt
10240000	Fdt
20480000	Fdt
40960000	Fdt
81920000	Fdt

java -Xint p11.Vector7 1

N	T
10000	0
20000	0
40000	0
80000	1
160000	2
320000	3
640000	6
1280000	13
2560000	24
5120000	48
10240000	99
20480000	194
40960000	385
81920000	761

java p11.Vector5 10

N	T
10000	1
20000	2
40000	1
80000	1
160000	0
320000	0
640000	2
1280000	5
2560000	8
5120000	16
10240000	33
20480000	62
40960000	126
81920000	250

java p11.Vector6 1

N	T
10000	42
20000	158
40000	623
80000	2508
160000	9955
320000	39820
640000	Fdt
1280000	Fdt
2560000	Fdt
5120000	Fdt
10240000	Fdt
20480000	Fdt
40960000	Fdt
81920000	Fdt

java p11.Vector7 1

N	T
10000	0
20000	0
40000	0
80000	0
160000	0
320000	0
640000	1
1280000	0
2560000	1
5120000	3
10240000	5
20480000	11
40960000	24
81920000	47

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?

-Si la función es O(n) se multiplicaría por 2, si es 0(n^2) se multiplicaría por 4

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para k=3 y k=4 y compruebe los tiempos obtenidos.)?

Pues en O(n) se multiplicaría por k y en O(n^2) se multiplicaría por k^2

¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal O(n)?

Si ya que va aproximadamente lineal y cada vez se multiplica por 2

Práctica 1.2

Ejercicio 1

Todos se han ejecutado con 1

N	TBucle1	TBucle2	TBucle3	TBucle4
100	0	0	1	1
200	0	1	4	6
400	1	3	15	58
800	0	14	64	359
1600	0	61	272	2832
3200	0	301	1182	20819
6400	1	1179	4847	169049
12800	3	4950	21273	FDT
25600	4	21881	88009	FDT
51200	7	79546	354744	FDT

Ejercicio 2

Todos se han ejecutado con 1

N	TBucle5	TBucle6	TBucle7
100	1	1	1
200	4	4	6
400	23	15	58
800	113	64	359
1600	522	272	2832
3200	2491	1182	20819
6400	11547	4847	169049
12800	53269	21273	FDT
25600	241669	88009	FDT
51200	FDT	354744	FDT

Ejercicio 3

N	TBucle1(t1)	TBucle2(t2)	T1/t2
100	0	1	0
200	0	1	0
400	1	3	0,33333333
800	0	14	0
1600	0	61	0
3200	0	301	0
6400	1	1179	0,00084818
12800	3	4950	0,00060606
25600	4	21881	0,00018281
51200	7	79546	8,7999E-05

Ejercicio 4

N	TBucle3(t3)	TBucle2(t2)	T3/t2
100	1	1	1
200	4	1	4
400	15	3	5
800	64	14	4,57142857
1600	272	61	4,45901639
3200	1182	301	3,9269103
6400	4847	1179	4,11111111
12800	21273	4950	4,29757576
25600	88009	21881	4,02216535
51200	354744	79546	4,45960828

Ejercicio 5

N	Bucle4-Python	Bucle4-Java sin JIT	tPython/tJava
100	1	1	1
200	4	1	4
400	15	3	5
800	64	14	4,57142857
1600	272	61	4,45901639
3200	1182	301	3,9269103
6400	4847	1179	4,11111111

Práctica 2

1)INTERCAMBIO DIRECTO o BURBUJA N = 1

n	T ordenado	T inverso	T aleatorio
1000	455	1090	886
2000	1724	4394	3447
4000	6747	17567	13821
8000	27317	69867	55271
16000	107874	278160	
32000	FDT	FDT	FDT
64000	FDT	FDT	FDT
128000	FDT	FDT	FDT

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada:

Si que concuerdan ya que los datos reflejan un comportamiento cuadrático, O(n^2). Con respecto a las diferencias de ordenación, el ordenado es más rápido ya que evalúa la condición if (a[j - 1] > a[j], pero nunca entra dentro, sin embargo en las otras ordenaciones si que entra y tarda más.

2)SELECCIÓN N = 1

n	T ordenado	T inverso	T aleatorio
1000	394	349	389
2000	1578	1401	1570
4000	6458	5616	6331
8000	25641	22053	25197
16000	102444	88122	100381
32000	FDT	FDT	FDT
64000	FDT	FDT	FDT
128000	FDT	FDT	FDT

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada:

Si que concuerdan ya que los datos reflejan un comportamiento cuadrático, O(n^2). Con respecto a las diferencias de ordenación, en este caso el número de comparaciones dentro del algoritmo es constante y no tiene ninguna relación con el orden del array, por tanto el resultado tiene sentido.

3)INSERCIÓN N = 1

n	T ordenado	T inverso	T aleatorio
1000	O	703	342

2000	0	2737	1340
4000	1	10966	5387
8000	1	43784	21581
16000	3	172767	86386
32000	6	FDT	FDT
64000	12	FDT	FDT
128000	25	FDT	FDT

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada:

En este algoritmo podemos ver que los valores en el array ordenado es muy rápido en comparación con los demás ya que es su mejor caso y su coste es lineal O(n), en cambio su caso medio y peor es cuadrático O(n^2) y se refleja en el resultado.

4)RAPIDO N = 1

n	T ordenado	T inverso	T aleatorio
1000	3	3	4
2000	5	6	8
4000	11	11	15
8000	23	26	35
16000	49	53	71
32000	99	110	152
64000	211	230	322
128000	440	485	705

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada:

Si que concuerdan ya que los datos reflejan un comportamiento cuasi-lineal, O(n*log n). Con respecto a las diferencias de ordenación, en este caso se debe a la elección del pivote, ya que en el algoritmo cogemos el elemento central y esto facilita tanto al inverso como al ordenado, mientras que en el aleatorio tarda más.

Proyecte, a partir de las complejidades y los datos de las tablas anteriores, ¿cuántos días (1 día= 861400.000 milisegundos) tardaría cada uno de los cuatro métodos (Burbuja, Selección, Inserción y Rápido) en ordenar un vector de n= 811920.000 (o sea, n=80000*24*26=80000*210), en el caso inicialmente en orden aleatorio?

Burbuja:

T = 55.271 ms *104.857.600 = 5.795.584.409.600 ms (aprox)

Días: 5.795.584.409.600 / 86.400.000 = 67.078,5 dias (apox 183 años)

Selección:

T = 25.197 ms * 104.857.600 = 2.642.096.947.200 ms (aprox).

Días: 2.642.096.947.200 / 86.400.000 = 30.579,8 dias (aprox. 83 años).

Inserción:

T = 21.581 ms * 104.857.600 = 2.262.931.865.600 ms (aprox).

Días: 2.262.931.865.600 / 86.400.000 = 26.191,3 dias (aprox. 71 años).

Rápido:

n=128.000 es K = 81.920.000 / 128.000 = 640.

T(n_2) = T(n_1) * (n1/n2) * (log_2(n2) / log_2(n1));

log_2(128.000) = 16,96 aprox

log_2(81.920.000) = 26,28 aprox

T = 705 ms * 640 * (26,28 / 16,96) = 699.114 ms.

Días: 699.114 / 86.400.000 = 0,008 aprox (**11,6 minutos**)

5)RÁPIDO vs. INSERCIÓN para tamaños bajos para n = 100000

n	T rapido	T insercion	T rápido/T insercion
10	122	61	2
15	189	110	1,71818182
20	285	183	1,55737705
25	361	246	1,46747967
30	419	364	1,1510989
35	562	492	1,14227642
40	620	644	0,96273292
45	736	777	0,94723295
50	815	939	0,86794462
55	925	1118	0,8273703
60	1046	1319	0,79302502
65	1104	1604	0,6882793
70	1230	1844	0,6670282
75	1367	2116	0,64603025
80	1472	2342	0,62852263
85	1555	2614	0,59487376
90	1751	2943	0,59497112
95	1830	3261	0,56117755
100	1906	3637	0,52405829

Razone a partir de qué valor de n comienza a tardar menos tiempo el algoritmo de menor complejidad (rápido) que el de mayor complejidad (inserción).

En la iteración 40 el algoritmo rápido ya es más veloz que el de inserción porque cada iteración el algoritmo rápido hace una partición y cada vez tiene que recorrer la mitad de números por lo tanto es más rápido

Práctica 3

EJ 1

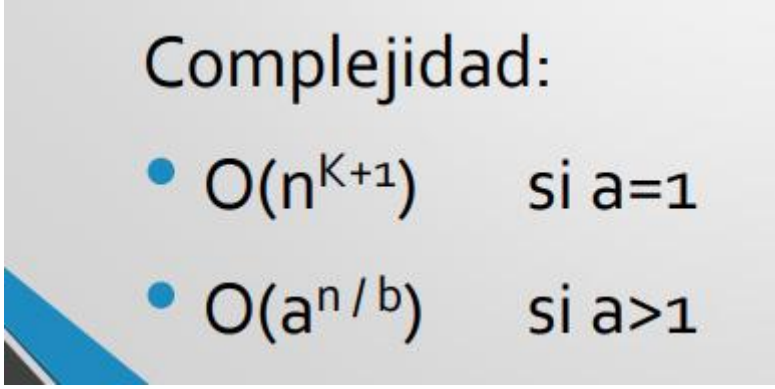
Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

Sustracción 1 O(n) -> a = 1, b = 1, k = 0

Sustracción 2 0(n^2) -> a = 1, b = 1, k = 1

Sustracción 3 O(2^n) -> a = 2, b = 1, k = 0

Las complejidades se sacan a través de la fórmula



Contestar: ¿para qué valor de n dejan de dar tiempo (abortan) las clases Sustracción1 y Sustracción2? ¿por qué sucede eso?

A partir de n 8000 sustracción 1 deja de dar tiempos

A partir de n 8000 sustracción 2 deja de dar tiempos

Esto sucede porque la pila de métodos de java se llena, ya que lo único que hacemos es delegar siempre en otro método

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción3 para n=80? Razonar la respuesta.

para n = 30 T=42,761 segundos

O(a^n/b) -> O(a^n) -> 0 (a^80) -> O(2 ^80-30) = O(2^50) (aprox)

T * 2^50 = 42,761 s * 2^50 = 4.8149×10^19 segundos

Segundos en un año = 3,1536 * 10^7 segundos

Años = 4.8149×10^19 segundos / 3,1536 * 10^7 segundos = 1.53×10^12 años (aporx)

Implementar una clase Sustracción4.java con una complejidad O(n3) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=100, 200, 400, 800, … (hasta FdT).

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++) {
            for (int j=0;j<n;j++) {
                cont++;
            }
        }

        rec4 (n-1);
    }
}
```

N	20	40	80	160	320	640	1280	2560	5120	10240
T	0	1	2	19	159	1246	9955	79593	636571	FDT

Implementar una clase Sustracción5.java con una complejidad O(3n/2) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=30, 32, 34, 36, … (hasta FdT).

```
public static void rec5 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        rec5 (n-2);
        rec5 (n-2);
        rec5 (n-2);
    }
}
```

N	28	30	32	34	36	38	40	42	44
T	174	517	1549	4788	14263	43154	387460	fdt	fdt

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción5 para n=80? Razonar la respuesta

T para n = 36 -> 14,263 s

O(3 ^ (80 - 36) / 2) * T para n= 36 -> O(3 ^ (80 - 36) / 2) * 14,263 s = 447588053203167 segundos 447588053203167 / 3153600 = 14192924,06 años

2)DyV por DIVISIÓN

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo

Division 1 O(n) -> a = 1, b = 3, k = 1

Division 2 O(n * log n) -> a = 2, b = 2, k = 1

Division 3 O(n^log_2 2) -> a = 2, b = 2, k = 0

La complejidad coincide con los tiempos y ha sido calculado a través del teorema maestro

Implementar una clase Division4.java con una complejidad O(n2) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, … (hasta FdT)

```
public static void rec4 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            for (int i=1;i<n;i++) {
                for (int j=1;j<n;j++) {
                    cont++ ;
                }
            }
            rec4 (n/2);
        }

    }
```

N	T

1000	**20**
2000	**74**
4000	**296**
8000	**1189**
16000	**4821**
32000	**19294**
64000	**77523**
128000	**FDT**
256000	**FDT**

Implementar una clase Division5.java con una complejidad O(n2) (con a>bk) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, … (hasta FdT).

```
public static void rec5 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            rec5 (n/2);
            rec5 (n/2);
        }

    }
```

N	T
1000	**0**
2000	**0**
4000	**0**
8000	**1**
16000	**1**
32000	**2**
64000	**4**
128000	**8**
256000	**15**
512000	**29**
1024000	**62**
2048000	**121**
4096000	**242**
8192000	**482**

3)DOS EJEMPLOS BÁSICOS

VectorSuma: (Realizado para n = 10)

Sum1 – O(n)

Sum2 – (a=1;b=1;k=0)=>O(n)

Sum3 – (a=2;b=2;k=0)=>O(n)

Sum1	Sum2	Sum3
888 NANOSEGUNDOS	2192 NANOSEGUNDOS	4575 NANOSEGUNDOS

Fibonacci:

Fib1 – 0(n)

Fib2 – O(n)

Fib3 – /* recursiva Sustraccion (a=1;b=1;k=0)=>O(n) */

Fib4 - /* recursiva Sustraccion (a=2;b=1 o b=2;k=0)=>O(1.6n)**

es intratable por exponencial */

Fib1	Fib2	Fib3	Fib4
428 NANOSEGUNDOS	580 NANOSEGUNDOS	868 NANOSEGUNDOS	6192 MILISEGUNDOS

4)PRÁCTICA DyV

p3p.PuntosTrivialTiempos.java

N	T (ms)
1024	10
2048	10
4096	14
8192	59
16384	229
32768	923
65536	3361
131072	13283
262144	55112
524288	225367

Razone si sintonizan los tiempos obtenidos con la complejidad del algoritmo.

Si que tienen sentido los resultados ya que la complejidad es O(n^2) y se corresponde.

p3p.PuntosDyVTiempos.java

N	T (ms)
1024	5
2048	2
4096	2
8192	4
16384	12
32768	31
65536	73
131072	119
262144	137
524288	305
1048576	688
2097152	1318
4194304	3369
8388608	8238
16777216	19454
33554432	45582

Razone si sintonizan los tiempos obtenidos con la complejidad del algoritmo.

Si que tienen sentido los resultados ya que la complejidad es O(n*logn) y se corresponde.

Práctica 4

Mediciones de tiempo para n = 1000

n	T Coloreado JAVA(ms)	T Coloreado PYTHON(ms)
4	6	3
8	5	7
16	12	16
32	21	41
64	38	93
128	49	173
256	103	378
512	138	865
1024	309	1654
2048	588	3223
4096	1369	6711
8192	3187	13640
16284	8738	39383
32768	18208	59154
65536	49329	130400

Práctica 5

ALGORITMO FERRY:

1. Definición del estado

Para resolver este problema mediante programación dinámica, utilizamos una matriz bidimensional donde N es el número total de vehículos y L es la longitud de los carriles del ferry.

El estado se define por los índices dp[i][j], cuyo significado es el siguiente:

- **i (Filas):** Representa el número de vehículos que se han procesado o intentado cargar hasta el momento.
- **j (Columnas):** Representa la longitud exacta ocupada en el carril de **Babor**.
- **Valor (True / False):** Indica si es posible acomodar los primeros i vehículos de forma que el carril de babor ocupe exactamente una longitud j$

2. Relación de recurrencia

Sea v_i la longitud del vehículo en la posición i y S_i la suma acumulada de las longitudes de los primeros i vehículos. La transición de estados se define de la siguiente manera:

Casos base:

- Para 0 vehículos procesados, la longitud ocupada es 0.

dp[0][0] = True

- Cualquier otra longitud con 0 vehículos es imposible.

dp[0][j] = False para todo j > 0

Recurrencia (Transición):

Para calcular el estado dp[i][j] a partir de la fila anterior (i-1), evaluamos si el vehículo i se puede añadir a Babor o a Estribor. El estado dp[i][j] será True si se cumple al menos una de las siguientes condiciones:

1. El vehículo i se coloca en Babor: Era posible alcanzar una longitud j - v_i con i-1 vehículos, y al sumar v_i no superamos la longitud máxima del ferry.
2. El vehículo i se coloca en Estribor: Era posible alcanzar la misma longitud j en Babor con i-1 vehículos, y al añadir el vehículo i al lado de Estribor, este no supera la longitud del ferry.

3.Complejidad

Tenemos dos bucles anidados, el primero de 0 a N vehículos y el segundo de 0 a L longitud del ferry. Por tanto es **O (N * L)**

Práctica 6

Backtracking

Resultado de tiempos para **p6.AlmacenajeContenedoresTiempos** con n = 100

Nombre Test	T(ms)
Test00	71
Test01	71
Test02	17
Test03	47
Test04	46
Test05	33
Test06	36
Test07	26
Test08	2511
Test09	FDT

P6-EXTRA DAVID RICO

Backtracking

Resultado de tiempos para NReinas

Tamaño de tablero (n)	T(ms)	Con -Xint
10	5	11
11	26	55
12	77	306
13	348	1785
14	1999	10632
15	12901	70000
16	Java heap space	Java heap space

Recursividad espacial = O(n) Recursividad Temporal = O()

Práctica 7

Ramifica y Poda

a)

Condición de poda: Si el número de contenedores que ya tenemos abiertos más ese mínimo teórico adicional es mayor o igual al número de contenedores de la mejor solución encontrada hasta ahora (mejorK), esa rama se corta inmediatamente. Es imposible que por ahí encontremos una solución mejor, ahorrando así miles de llamadas recursivas

Resultado de tiempos para **p6.AlmacenajeContenedoresRyPTiempos** con n = 1000

Nombre Test	T(ms)
Test00	1838
Test01	1748
Test02	1812
Test03	1711
Test04	1839
Test05	1912
Test06	1914
Test07	1885
Test08	1929
Test09	1933